

# Machine Learning Exam - Comprehensive Question Bank

## PART A: INTRODUCTION TO MACHINE LEARNING

### Mathematical Questions

**Q1.** Given a machine learning system with experience E, task T, and performance measure P. If the system's performance improves from 65% to 82% accuracy after processing 10,000 additional samples, calculate:

- a) The performance improvement rate per 1000 samples
- b) If the improvement follows a logarithmic curve  $P(n) = a \cdot \log(n) + b$ , estimate parameters a and b given  $P(1000) = 65\%$  and  $P(11000) = 82\%$

**Answer:**

- a) Improvement =  $82\% - 65\% = 17\%$  over 10,000 samples = 1.7% per 1000 samples
- b)  $65 = a \cdot \log(1000) + b$  and  $82 = a \cdot \log(11000) + b$ 
  - Solving:  $17 = a \cdot (\log(11000) - \log(1000)) = a \cdot \log(11)$
  - $a = 17/\log(11) \approx 16.36$ ,  $b = 65 - 16.36 \cdot \log(1000) \approx 15.93$

### Conceptual Questions

**Q2.** Explain why Tom Mitchell's definition requires all three components (E, T, P) for machine learning. What happens if any component is missing?

**Answer:** All three are essential:

- **E (Experience):** Without data/experience, there's nothing to learn from - it's just hardcoded logic
- **T (Task):** Without a defined task, there's no objective or direction for learning
- **P (Performance):** Without measurable performance, we cannot verify if learning occurred or guide improvement Missing any component means either no learning happens or we cannot verify/measure it.

**Q3.** Differentiate between supervised, unsupervised, and reinforcement learning with respect to:

- Data structure
- Learning objective
- Feedback mechanism

**Answer:**

| Aspect    | Supervised                         | Unsupervised                | Reinforcement               |
|-----------|------------------------------------|-----------------------------|-----------------------------|
| Data      | (x, y) pairs with labels           | Only x, no labels           | Sequential states & actions |
| Objective | Learn $f: X \rightarrow Y$ mapping | Discover patterns/structure | Maximize cumulative reward  |
| Feedback  | Direct (correct labels)            | None (self-discovered)      | Delayed (reward signals)    |

## Scenario-Based Questions

**Q4.** You're building a spam email classifier. After training on 5000 emails:

- Training accuracy: 95%
- Validation accuracy: 88%
- Test accuracy: 87%

Your manager asks why we need three separate datasets. Explain the role of each and what the accuracy differences indicate.

**Answer:**

- **Training Set:** Used to learn parameters. 95% shows model fits training data well.
- **Validation Set:** Used for hyperparameter tuning and model selection. 88% suggests some overfitting (7% drop).
- **Test Set:** Final unbiased evaluation. 87% is true expected performance on unseen data.

The gap between training (95%) and validation/test (87-88%) indicates mild overfitting. We need separate sets to detect this and ensure our model generalizes rather than memorizing.

## Intuitive Questions

**Q5.** Why is clustering considered "unsupervised" when we still provide the number of clusters  $k$ ? Isn't that a form of supervision?

**Answer:** Supervision refers to providing **labeled outputs/targets** for each input. In clustering:

- We don't tell the algorithm which samples belong together
- $k$  is a **hyperparameter** (structural choice), not a label
- The algorithm discovers patterns and groupings independently
- It's like saying "organize these into 5 groups" vs "this belongs in group A, that in group B" (supervised)

**Q6.** In Tom Mitchell's definition, can "performance measure P" ever decrease with more experience E? When might this happen?

**Answer:** Theoretically no, but practically yes in cases like:

- **Concept drift:** Data distribution changes over time (old experience becomes harmful)
- **Overfitting:** Model memorizes training noise, performance on new data degrades
- **Catastrophic forgetting:** Learning new tasks destroys knowledge of old tasks
- **Adversarial attacks:** Malicious data poisoning degrades performance True learning should improve P, but poor algorithms or changing environments can violate this.

**Related Topic Questions**

**Q7.** Compare and contrast Classification vs Regression vs Clustering:

- Output space
- Loss functions typically used
- Evaluation metrics
- Example applications

**Answer:**

| Aspect  | Classification          | Regression        | Clustering                       |
|---------|-------------------------|-------------------|----------------------------------|
| Output  | Discrete classes        | Continuous values | Group assignments                |
| Loss    | Cross-entropy, 0-1 loss | MSE, MAE, Huber   | Within-cluster variance          |
| Metrics | Accuracy, F1, AUC       | RMSE, R², MAE     | Silhouette score, Davies-Bouldin |
| Example | Spam detection          | House price       | Customer segmentation            |

**PART B: LINEAR REGRESSION**

**Mathematical Questions**

**Q8.** Derive the normal equation for linear regression starting from the cost function  $J(\theta) = (1/2n)\sum(h_{\theta}(x^{(i)}) - y^{(i)})^2$ . Show all steps.

**Answer:**

$$J(\theta) = (1/2n) \|X\theta - y\|^2$$

$$\partial J / \partial \theta = (1/n) X^T (X\theta - y) = 0$$

$$X^T X \theta = X^T y$$

$$\theta^* = (X^T X)^{-1} X^T y$$

**Q9.** Given a dataset with  $n=4$  samples and  $d=2$  features:

$$X = [1, 2; 1, 3; 1, 4; 1, 5] \text{ (with } x_0=1\text{)}$$

$$y = [3; 5; 7; 9]$$

Calculate  $\theta^*$  using the normal equation.

**Answer:**

$$X^T X = [4, 14; 14, 54]$$

$$X^T y = [24; 86]$$

$$(X^T X)^{-1} = (1/20) [54, -14; -14, 4] = [2.7, -0.7; -0.7, 0.2]$$

$$\theta^* = [2.7, -0.7; -0.7, 0.2] [24; 86] = [4.2 - 60.2; -16.8 + 17.2]$$

$$\theta^* = [1; 2]$$

$$\text{So } h(x) = 1 + 2x_1$$

**Q10.** For linear regression, prove that the cost function  $J(\theta) = (1/2n) \sum (\theta^T x^{(i)} - y^{(i)})^2$  is convex.

**Answer:** A function is convex if its Hessian  $H$  is positive semi-definite.

$$J(\theta) = (1/2n) (X\theta - y)^T (X\theta - y)$$

$$\nabla J = (1/n) X^T (X\theta - y)$$

$$H = \nabla^2 J = (1/n) X^T X$$

$$\text{For any vector } v: v^T H v = (1/n) v^T X^T X v = (1/n) \|Xv\|^2 \geq 0$$

Therefore  $H$  is PSD, and  $J(\theta)$  is convex.

## Conceptual Questions

**Q11.** Why do we use  $(1/2n)$  instead of  $(1/n)$  in the cost function? Does it affect the optimal  $\theta^*$ ?

**Answer:**

- The  $1/2$  simplifies derivative calculation (cancels with power of 2)
- Doesn't affect  $\theta^*$  because we're optimizing (finding where derivative = 0)
- Constant factors don't change the location of minimum, only scale the loss value

- Convention for mathematical convenience

**Q12.** When would  $(X^T X)$  be non-invertible? Provide at least 3 scenarios and their implications.

**Answer:**

1. **Multicollinearity:** Features are linearly dependent (e.g.,  $x_2 = 2x_1$ )
  - Matrix is rank deficient
  - Infinitely many solutions exist
2.  **$n < d$ :** More features than samples
  - Underdetermined system
  - Need regularization (Ridge/Lasso)
3. **Duplicate features:** Same column repeated
  - Redundant information
  - Remove duplicates

**Implications:** Use pseudoinverse, regularization, or feature selection.

### Scenario-Based Questions

**Q13.** You're predicting house prices using linear regression:

- Features: area (500-5000 sq ft), bedrooms (1-5), age (0-100 years)
- After training, you get:  $\theta = [50000, 100, -20000, -500]$

Interpret each coefficient and explain if this model makes practical sense.

**Answer:**

- **$\theta_0 = 50,000$ :** Base price (intercept) - a house with 0 area/bedrooms/age costs \$50k (unrealistic baseline)
- **$\theta_1 = 100$ :** Each sq ft adds \$100 (reasonable)
- **$\theta_2 = -20,000$ :** Each bedroom **reduces** price by \$20k (illogical! Should be positive)
- **$\theta_3 = -500$ :** Each year of age reduces price by \$500 (reasonable - depreciation)

**Problem:** Negative coefficient for bedrooms suggests multicollinearity or poor feature scaling. Bedrooms likely correlates with area, causing instability.

**Q14.** Your linear regression model achieves:

- Training MSE: 0.05
- Test MSE: 4.5

What's happening and how would you address it?

**Answer: Problem:** Severe overfitting (90x higher test error)

**Causes:**

- Model too complex for data
- Training set too small
- Features not representative

**Solutions:**

1. Regularization (Ridge/Lasso)
2. Feature selection/reduction
3. Collect more training data
4. Check for outliers
5. Cross-validation for hyperparameter tuning

**Intuitive Questions**

**Q15.** Why is it called "linear" regression when  $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2$  can model a parabola?

**Answer:** "Linear" refers to linearity **in parameters  $\theta$** , not in input features:

- $h(x)$  is a **linear combination** of  $\theta$  values
- Even if  $x_1^2$  is a feature, it's still  $\theta_0 \cdot 1 + \theta_1 \cdot x_1 + \theta_2 \cdot x_1^2$
- Each  $\theta$  appears with power 1 (linear in  $\theta$ )
- This allows us to use linear algebra (normal equation works)
- Contrast with  $\theta_1^{x_1}$  (exponential) - that's non-linear in parameters

**Q16.** If you have a perfect fit ( $J(\theta) = 0$ ) on training data, is your model guaranteed to perform well on test data? Why or why not?

**Answer: No**, for several reasons:

1. **Overfitting:** Model memorized training noise, won't generalize
2. **Train/test distribution mismatch:** Test data from different distribution
3. **Small training set:** Perfect fit might be coincidence
4. **High model complexity:** Can always fit  $n$  points with degree  $n-1$  polynomial

Perfect training fit often indicates overfitting unless:

- Data is truly deterministic (no noise)
- Model captures true underlying relationship
- Training set is large and representative

## Related Topic Questions

**Q17.** How would you modify linear regression to handle:

- a) Outliers that heavily influence the solution
- b) Multicollinearity between features
- c) Non-linear relationships

**Answer: a) Outliers:**

- Use robust loss functions (Huber loss, MAE instead of MSE)
- Outlier detection and removal
- Weighted least squares

**b) Multicollinearity:**

- Ridge regression (L2 regularization)
- Feature selection/PCA
- Remove redundant features
- Increase training data

**c) Non-linear relationships:**

- Feature engineering (polynomial features, log transforms)
- Kernel methods
- Switch to non-linear models (neural networks, decision trees)

---

## PART C: GRADIENT DESCENT

### Mathematical Questions

**Q18.** For  $J(\theta) = (1/2)(\theta_1^2 + 4\theta_2^2)$ , perform 3 iterations of gradient descent starting from  $\theta = [2, 2]^T$  with  $\alpha = 0.1$ . Calculate the exact  $\theta$  values.

**Answer:**

$$\nabla J = [\theta_1; 8\theta_2]$$

Iteration 0:  $\theta = [2, 2]$

$$\nabla J = [2; 16]$$

$$\theta_1 = 2 - 0.1(2) = 1.8$$

$$\theta_2 = 2 - 0.1(16) = 0.4$$

Iteration 1:  $\theta = [1.8, 0.4]$

$$\nabla J = [1.8; 3.2]$$

$$\theta_1 = 1.8 - 0.1(1.8) = 1.62$$

$$\theta_2 = 0.4 - 0.1(3.2) = 0.08$$

Iteration 2:  $\theta = [1.62, 0.08]$

$$\nabla J = [1.62; 0.64]$$

$$\theta_1 = 1.62 - 0.1(1.62) = 1.458$$

$$\theta_2 = 0.08 - 0.1(0.64) = 0.016$$

**Q19.** Derive the update rule for gradient descent in linear regression. Show that:

$$\theta_j := \theta_j - \alpha(1/n)\sum_i(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

**Answer:**

$$J(\theta) = (1/2n)\sum_i(h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\begin{aligned}\partial J / \partial \theta_j &= (1/2n)\sum_i 2(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot \partial h_{\theta} / \partial \theta_j \\ &= (1/n)\sum_i(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}\end{aligned}$$

$$\begin{aligned}\text{Update: } \theta_j &:= \theta_j - \alpha \cdot \partial J / \partial \theta_j \\ &= \theta_j - \alpha(1/n)\sum_i(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}\end{aligned}$$

**Q20.** For mini-batch SGD with batch size  $m=32$  and dataset size  $n=1000$ , calculate:

- a) Number of mini-batches per epoch
- b) Number of updates per epoch
- c) If using 10 epochs, total number of gradient computations

**Answer:**

- a)  $1000/32 = 31.25 \rightarrow 31$  mini-batches (last incomplete batch dropped)



- b) 31 updates per epoch
- c) 10 epochs  $\times$  31 updates = 310 gradient computations

## Conceptual Questions

**Q21.** Compare Batch GD, Stochastic GD, and Mini-batch GD on:

- Gradient accuracy
- Convergence stability
- Memory requirements
- Computational speed
- Parallelization capability

**Answer:**

| Aspect            | Batch GD              | SGD                       | Mini-batch GD       |
|-------------------|-----------------------|---------------------------|---------------------|
| Gradient accuracy | Exact                 | Very noisy                | Moderately noisy    |
| Convergence       | Smooth, deterministic | Oscillates, never settles | Reasonably stable   |
| Memory            | High (all n samples)  | Low (1 sample)            | Medium (m samples)  |
| Speed per update  | Slow                  | Very fast                 | Fast                |
| Parallelization   | Limited               | None                      | Good (GPU-friendly) |
| Use case          | Small datasets        | Online learning           | Standard choice     |

**Q22.** Explain why mini-batch SGD gradient is an unbiased estimator of the true gradient. What does "unbiased" mean and why is it important?

**Answer: Unbiased** means  $E[\text{estimate}] = \text{true value}$ .

For mini-batch gradient  $g_B$ :

$$\begin{aligned}
 E[g_B] &= E[(1/m) \sum_{i \in B} \nabla J(\theta, x^{(i)}, y^{(i)})] \\
 &= (1/m) \sum_{i \in B} E[\nabla J(\theta, x^{(i)}, y^{(i)})] \\
 &= (1/m) \cdot m \cdot (1/n) \sum_{\text{all } i} \nabla J(\theta, x^{(i)}, y^{(i)}) \\
 &= \nabla J(\theta) \quad [\text{true gradient}]
 \end{aligned}$$

**Importance:**

- On average, we move in the correct direction

- Convergence guarantees hold
- Noise averages out over iterations
- Though variance exists, expectation is correct

## Scenario-Based Questions

**Q23.** You're training a model with 1 million samples. After 1 epoch:

- Batch GD: 2 hours, loss = 0.5
- SGD: 10 minutes, loss = 0.8
- Mini-batch (size=64): 15 minutes, loss = 0.52

Which should you choose and why? Consider you have limited time.

**Answer: Choose Mini-batch GD** because:

### 1. **Best trade-off:**

- Loss (0.52) nearly as good as Batch GD (0.5)
- 8x faster than Batch GD (15 min vs 2 hours)
- More stable than SGD (0.52 vs 0.8)

### 2. **Scalability:**

- Can train many epochs in Batch GD's 1 epoch time
- GPU-friendly parallelization
- After 8 epochs (2 hours), likely beats Batch GD

### 3. **SGD issues:**

- Much higher loss (0.8)
- May never converge to good solution
- Too noisy for stable training

**Q24.** Your gradient descent diverges (loss increases). You suspect learning rate.  $\alpha$  is currently 0.1. Describe your debugging process.

**Answer:**

### **Step 1 - Confirm divergence:**

- Plot loss vs iterations
- Check if gradients exploding (print gradient norms)

## Step 2 - Test smaller learning rates:

- Try  $\alpha = [0.01, 0.001, 0.0001]$
- Use learning rate finder (start small, gradually increase)

## Step 3 - Check implementation:

- Verify gradient computation (numerical gradient check)
- Ensure correct update rule ( $\theta := \theta - \alpha \nabla J$ , not +)
- Check for bugs in loss calculation

## Step 4 - Investigate other causes:

- Poor feature scaling (normalize features)
- Exploding gradients (gradient clipping)
- Bad initialization (Xavier/He initialization)
- Non-convex with bad local minimum

## Step 5 - Solutions:

- Learning rate schedule (decay over time)
- Adaptive optimizers (Adam, RMSprop)
- Gradient clipping
- Better preprocessing

## Intuitive Questions

**Q25.** If learning rate is "too large," loss might increase. If "too small," training is slow. Can you be more precise about what "too large" and "too small" mean mathematically?

**Answer:**

**Too Large** ( $\alpha > 2/L$  where  $L$  is Lipschitz constant):

- Step size exceeds distance to minimum
- Overshoots and bounces back further
- For quadratic  $J(\theta) = (1/2)\theta^T A \theta$ , need  $\alpha < 2/\lambda_{\max}(A)$
- Symptoms: loss oscillates or increases

**Too Small** ( $\alpha \ll \text{optimal}$ ):

- Makes progress but inefficiently
- Takes many iterations:  $\theta$  moves by tiny increments
- Time to convergence  $\sim 1/\alpha$
- Symptoms: very slow training, gets stuck

### Goldilocks zone:

- For strongly convex:  $\alpha \approx 1/L$  gives linear convergence
- Adaptive methods (Adam) automatically adjust
- Rule of thumb: start at 0.001-0.01, tune by factor of 10

**Q26.** Why does SGD sometimes escape local minima better than batch GD, even though it's noisier?

**Answer:**

### Noise as a feature:

1. **Stochasticity adds exploration:** Random fluctuations can push out of shallow local minima
2. **Like simulated annealing:** Random jumps help escape bad regions
3. **Smoother loss landscape:** Batch GD sees true (possibly non-convex) loss; SGD sees noisy approximation that might "blur out" small local minima

### Batch GD problems:

- Deterministic path gets stuck in local minima
- No mechanism to escape once gradient  $\approx 0$
- Needs random restarts or momentum

### However:

- Modern deep learning: local minima often not a problem (loss landscape is well-behaved)
- Noise helps early training, hurts final convergence
- Solution: Start noisy (SGD), then reduce noise (larger batches) or learning rate

### Related Topic Questions

**Q27.** How do momentum-based optimizers (SGD with momentum, Nesterov) differ from vanilla gradient descent? Explain the intuition and update rules.

**Answer:**

**Vanilla GD:**  $\theta := \theta - \alpha \nabla J$

- Only current gradient matters
- Changes direction instantly

**SGD with Momentum:**

$$v := \beta v + \nabla J$$

$$\theta := \theta - \alpha v$$

- **Intuition:** Rolling ball accumulates velocity
- **Benefits:**
  - Accelerates in consistent directions
  - Dampens oscillations
  - Escapes plateau regions faster
- $\beta \in [0,1]$ : typically 0.9, controls memory of past gradients

**Nesterov Momentum:**

$$v := \beta v + \nabla J(\theta - \alpha \beta v) \text{ [look-ahead gradient]}$$

$$\theta := \theta - \alpha v$$

- **Intuition:** Look ahead before computing gradient (corrective behavior)
- **Benefits:** More responsive to changes, better convergence
- **Anticipatory:** Reduces overshooting

**Comparison:**

- Momentum: "Move first, look later"
- Nesterov: "Look first, then move"
- Both: Dramatically faster than vanilla GD on ravines/valleys

---

## PART D: LOGISTIC REGRESSION

### Mathematical Questions

**Q28.** Derive the sigmoid function starting from odds and log-odds. Show why  $\sigma(z) = 1/(1+e^{(-z)})$  is the natural

choice for binary classification.

**Answer:**

Given:  $p = P(y=1|x)$ ,  $q = 1-p = P(y=0|x)$

Odds:  $p/q = p/(1-p)$

Log-odds:  $\log(p/(1-p)) = z = \theta^T x$  [assume linear]

Solving for  $p$ :

$$p/(1-p) = e^z$$

$$p = (1-p)e^z$$

$$p = e^z - pe^z$$

$$p(1 + e^z) = e^z$$

$$p = e^z/(1+e^z) = 1/(1+e^{-z}) = \sigma(z)$$

Why natural:

- Maps  $\mathbb{R} \rightarrow (0,1)$  smoothly
- Symmetric:  $\sigma(-z) = 1 - \sigma(z)$
- Nice derivative:  $\sigma'(z) = \sigma(z)(1-\sigma(z))$
- Probabilistic interpretation via logistic distribution

**Q29.** Prove that the derivative of sigmoid is  $\sigma'(z) = \sigma(z)(1-\sigma(z))$ .

**Answer:**

$$\sigma(z) = 1/(1+e^{-z})$$

Using quotient rule or rewrite:

$$\sigma(z) = (1+e^{-z})^{-1}$$

$$\begin{aligned}\sigma'(z) &= -(1+e^{-z})^{-2} \cdot (-e^{-z}) \\ &= e^{-z}/(1+e^{-z})^2\end{aligned}$$

Factor out from numerator and denominator:

$$\begin{aligned}&= (e^{-z}/(1+e^{-z})) \cdot (1/(1+e^{-z})) \\ &= (1 - 1/(1+e^{-z})) \cdot 1/(1+e^{-z}) \\ &= (1 - \sigma(z)) \cdot \sigma(z) \\ &= \sigma(z)(1 - \sigma(z)) \quad \checkmark\end{aligned}$$

**Q30.** For logistic regression, derive the gradient of negative log-likelihood:

$$L(\theta) = -\sum_i [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1-y^{(i)}) \log(1-h_{\theta}(x^{(i)}))]$$

Show that  $\partial L / \partial \theta_j = \sum_i (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$

## Answer:

For single sample:

$$l = -[y \cdot \log(\sigma(z)) + (1-y) \cdot \log(1-\sigma(z))] \text{ where } z = \theta^T x$$

$$\begin{aligned} \partial l / \partial z &= -[y \cdot \sigma'(z) / \sigma(z) - (1-y) \cdot \sigma'(z) / (1-\sigma(z))] \\ &= -[y \cdot \sigma(1-\sigma) / \sigma - (1-y) \cdot \sigma(1-\sigma) / (1-\sigma)] \\ &= -[y(1-\sigma) - (1-y)\sigma] \\ &= -[y - y\sigma - \sigma + y\sigma] \\ &= -(y - \sigma) \\ &= \sigma(z) - y \\ &= h_{\theta}(x) - y \end{aligned}$$

$$\partial l / \partial \theta_j = (\partial l / \partial z)(\partial z / \partial \theta_j) = (h_{\theta}(x) - y) \cdot x_j$$

For all samples:

$$\partial L / \partial \theta_j = \sum_i (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

## Conceptual Questions

**Q31.** Why do we maximize likelihood instead of minimizing error rate (0-1 loss) in logistic regression?

## Answer:

### Problems with 0-1 loss:

1. **Non-differentiable:** Gradient is 0 everywhere or undefined
  - Can't use gradient descent
  - No information about "how wrong" we are
2. **Non-convex:** Many local minima
3. **Discrete jumps:** Small parameter changes don't change loss until prediction flips

### Benefits of log-likelihood:

1. **Smooth and differentiable:** Nice gradients for optimization
2. **Convex:** Guaranteed global minimum
3. **Probabilistic interpretation:** Maximum Likelihood Estimation principle
4. **Surrogate loss:** Approximates 0-1 loss but optimizable
5. **Calibrated probabilities:** Not just classification, but confidence

**Connection:** Minimizing negative log-likelihood  $\approx$  minimizing classification error, but with better optimization properties.

**Q32.** Explain the difference between Maximum Likelihood Estimation (MLE) and Maximum A Posteriori (MAP) estimation. When is MAP better?

**Answer:**

**MLE:**  $\theta_{\text{MLE}} = \text{argmax } P(D|\theta)$

- Frequentist approach
- $\theta$  is fixed but unknown
- Only uses data likelihood
- No prior beliefs

**MAP:**  $\theta_{\text{MAP}} = \text{argmax } P(\theta|D) = \text{argmax } P(D|\theta)P(\theta)$

- Bayesian approach
- $\theta$  is random variable
- Combines likelihood  $\times$  prior
- Incorporates prior knowledge

**When MAP is better:**

1. **Small datasets:** Prior prevents overfitting
2. **Prior knowledge exists:** Use domain expertise
3. **Regularization:** Prior acts as regularizer (e.g., Gaussian prior  $\rightarrow$  L2 regularization)
4. **Uncertainty quantification:** Full posterior distribution

**Example:**

- MLE: Flip coin 3 times, get 3 heads  $\rightarrow \theta_{\text{MLE}} = 1.0$  (unrealistic)
- MAP: With prior  $\text{Beta}(2,2) \rightarrow \theta_{\text{MAP}} \approx 0.71$  (more reasonable)

### Scenario-Based Questions

**Q33.** You trained a logistic regression model for disease diagnosis. For a patient, your model outputs  $h_{\theta}(x) = 0.55$ . Your colleague says "It's positive since  $>0.5$ " but the doctor says "Too uncertain, run more tests." Who's right and why?

**Answer:**

**Both perspectives valid:**

**Colleague (mathematical):**



- Threshold 0.5 maximizes accuracy when classes balanced
- $0.55 > 0.5 \rightarrow$  predict positive
- Technically correct classification rule

### **Doctor (practical):**

- Medical decisions have asymmetric costs
- False negative (miss disease): patient dies
- False positive (wrong diagnosis): unnecessary treatment
- 0.55 is only 5% confidence margin!

### **Better approach:**

1. **Adjust threshold:** Use 0.3 or 0.4 for medical diagnosis (favor sensitivity over specificity)
2. **Confidence intervals:**  $0.55 \pm$  uncertainty estimate
3. **Cost-sensitive learning:** Weight false negatives higher during training
4. **Risk analysis:** Expected cost =  $P(\text{FN}) \times \text{Cost}(\text{FN}) + P(\text{FP}) \times \text{Cost}(\text{FP})$

**Answer:** In high-stakes domains, 0.55 is too uncertain. Should adjust threshold or get more features.

**Q34.** Your binary classifier achieves:

- Training accuracy: 99%
- Test accuracy: 98%
- But data is 95% class 0, 5% class 1

Is this a good model? How would you properly evaluate it?

**Answer:**

**No, poor model!**

**Problem:** Baseline (always predict class 0) gives 95% accuracy. Model only 3-4% better than doing nothing.

**Proper evaluation:**

1. **Confusion Matrix:**

|        | Predicted |              |
|--------|-----------|--------------|
|        | 0         | 1            |
| Actual | 0         | [TN]    [FP] |
|        | 1         | [FN]    [TP] |

## 2. Better metrics:

- **Sensitivity/Recall:**  $TP/(TP+FN)$  - how many actual positives caught
- **Specificity:**  $TN/(TN+FP)$  - how many actual negatives caught
- **Precision:**  $TP/(TP+FP)$  - of predicted positives, how many correct
- **F1-Score:** Harmonic mean of precision and recall
- **AUC-ROC:** Area under ROC curve (threshold-independent)

## 3. Class-specific accuracy:

- Accuracy on class 0 samples
- Accuracy on class 1 samples (likely very poor)

## 4. Actions:

- Rebalance training data (oversampling minority, undersampling majority)
- Use class weights in loss function
- Change threshold (maybe predict 1 if  $h(x) > 0.3$ )
- Try more sophisticated models

## Intuitive Questions

**Q35.** Why does logistic regression assume a linear decision boundary when the sigmoid is clearly non-linear?

**Answer:**

**Key insight:** Decision boundary is where  $P(y=1) = P(y=0) = 0.5$

$$h_{\theta}(x) = \sigma(\theta^T x) = 0.5$$

$$\Leftrightarrow \theta^T x = 0 \quad [\text{since } \sigma(0) = 0.5]$$

This is a **hyperplane** (linear)!

## Non-linearity of sigmoid:

- Maps  $\theta^T x \rightarrow$  probabilities (0,1)
- Provides smooth probability estimates
- Makes loss function differentiable

**But decision boundary:**

- Just the set  $\{x : \theta^T x = 0\}$
- Completely linear in  $x$
- Sigmoid doesn't curve the boundary, just the probabilities

**To get non-linear boundaries:**

- Add polynomial features:  $x_1, x_2, x_1$